

# Detección de Anomalías Utilizando Autocodificadores

Autor: Matías Cisnero

Tutora: Dra. M. Juliana Gambini

Diciembre 2025

Universidad Nacional de Hurlingham





# Índice General

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>1. Introducción</b>                             | <b>11</b> |
| 1.1. Motivación . . . . .                          | 11        |
| 1.2. Estado del Arte . . . . .                     | 11        |
| <b>2. Materiales y Métodos</b>                     | <b>15</b> |
| 2.1. Conjunto de datos y AED . . . . .             | 15        |
| 2.2. Descripción de los métodos . . . . .          | 20        |
| 2.2.1. Autocodificadores . . . . .                 | 20        |
| 2.2.2. Métricas de evaluación del modelo . . . . . | 25        |
| <b>3. Resultados</b>                               | <b>29</b> |
| <b>4. Conclusiones</b>                             | <b>35</b> |



# Índice de Figuras

|                                                                                                                                      |    |
|--------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1. Ejemplo de un autocodificador, donde se codifica y decodifica una imagen (tomado de [1]).                                       | 15 |
| 2.2. Boxplot combinado de <i>Area Mean</i> , <i>Fractal Dimension Mean</i> y <i>Simmetry Mean</i> , segmentado por (normal/anómalo). | 17 |
| 2.3. Matriz de correlación entre los atributos del conjunto de datos.                                                                | 18 |
| 2.4. Esquema de la división del conjunto de datos.                                                                                   | 20 |
| 2.5. Arquitectura general de un autocodificador.                                                                                     | 21 |
| 2.6. Funciones de activación ReLU y GELU.                                                                                            | 23 |
| 3.1. Distribución del error de reconstrucción para el autocodificador A.                                                             | 30 |
| 3.2. Distribución del error de reconstrucción para el autocodificador B.                                                             | 30 |
| 3.3. Distribución del error de reconstrucción para el autocodificador C.                                                             | 31 |
| 3.4. Curvas ROC y AUC obtenidos para los modelos AE, SAE y CAE entrenados con el Conjunto A (0 % de anomalías).                      | 32 |
| 3.5. Curvas ROC y AUC obtenidos para los modelos AE, SAE y CAE entrenados con el Conjunto B (10 % de anomalías).                     | 32 |
| 3.6. Curvas ROC y AUC obtenidos para los modelos AE, SAE y CAE entrenados con el Conjunto C (25 % de anomalías).                     | 33 |



# Índice de Tablas

|      |                                                                                                                                                    |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1. | Descripción de las diez características base extraídas de las imágenes digitalizadas de FNA.                                                       | 16 |
| 2.2. | Distribución de la variable objetivo <i>diagnosis</i>                                                                                              | 17 |
| 2.3. | División del conjunto original en entrenamiento, validación y prueba, manteniendo la proporción original de clases.                                | 19 |
| 2.4. | Subconjuntos derivados del subconjunto de entrenamiento para evaluar la resistencia del autocodificador ante diferentes proporciones de anomalías. | 20 |
| 2.5. | Matriz de confusión. La diagonal verde representa predicciones correctas, mientras que las celdas rojas corresponden a errores de clasificación.   | 26 |
| 3.1. | Hiperparámetros de entrenamiento para cada modelo.                                                                                                 | 29 |
| 3.2. | Valores de umbral $\epsilon$ obtenidos para cada modelo.                                                                                           | 30 |
| 3.3. | Métricas de rendimiento para cada modelo entrenado con los conjuntos A, B y C.                                                                     | 31 |

## Glosario

| Notación                                   | Descripción                                                                                   |
|--------------------------------------------|-----------------------------------------------------------------------------------------------|
| $\mathbf{X} = \{x^{(1)}, \dots, x^{(N)}\}$ | Conjunto de datos (N ejemplos)                                                                |
| $X_i$                                      | Atributo $i$ -ésimo del conjunto de datos                                                     |
| $x^{(u)} \in \mathbb{R}^D$                 | Ejemplo $u$ -ésimo del conjunto de datos                                                      |
| $x_k^{(u)}$                                | Componente $k$ del ejemplo $x^{(u)}$                                                          |
| $\hat{x}^{(u)}$                            | Reconstrucción del ejemplo $x^{(u)}$                                                          |
| $z^{(u)} \in \mathbb{R}^K$                 | Representación latente del ejemplo $u$                                                        |
| $z_j^{(u)}$                                | Activación del componente $j$ de la representación latente                                    |
| $D$                                        | Dimensión del vector de entrada                                                               |
| $K$                                        | Dimensión de la capa latente                                                                  |
| $H$                                        | Dimensión de una capa oculta genérica                                                         |
| $N$                                        | Cantidad total de ejemplos                                                                    |
| $W$                                        | Conjunto de matrices de pesos                                                                 |
| $w_{jk}$                                   | Peso que conecta la entrada $k$ con la neurona $j$                                            |
| $b$                                        | Conjunto de vectores sesgo o <i>bias</i>                                                      |
| $b_j$                                      | Sesgo o <i>bias</i> de la neurona $j$                                                         |
| $a_j^{(u)}$                                | Activación de la neurona $j$ para el ejemplo $u$                                              |
| $g(x)$                                     | Función de activación                                                                         |
| $\mathcal{L}(W, b)$                        | Función de costo (MSE)                                                                        |
| $\bar{x}_i$                                | Media muestral del atributo $X_i$                                                             |
| $s_{x_i}$                                  | Desvío estándar muestral del atributo $X_i$                                                   |
| $\text{Cov}(X_i, X_j)$                     | Covarianza entre atributos $X_i$ y $X_j$                                                      |
| $\text{Corr}(X_i, X_j)$                    | Correlación entre atributos $X_i$ y $X_j$                                                     |
| $\Sigma \in \mathbb{R}^{D \times D}$       | Matriz de correlaciones                                                                       |
| $\rho$                                     | Activación promedio deseada (SAE)                                                             |
| $\hat{\rho}_j$                             | Activación promedio real de la neurona $j$ (SAE)                                              |
| $\text{KL}(p \parallel q)$                 | Divergencia Kullback–Leibler                                                                  |
| $\mathcal{L}_{\text{sparse}}(W, b)$        | Función de costo por dispersidad (SAE)                                                        |
| $J_f(x^{(u)})$                             | Jacobiano del codificador evaluado en $x^{(u)}$ (CAE)                                         |
| $\ A\ _F^2$                                | Cuadrado de la norma de Frobenius de la matriz $A$                                            |
| $\mathcal{L}_{\text{CAE}}(W, b)$           | Función de costo total del CAE (reconstrucción + penalización contractiva)                    |
| $\mathbf{T}$                               | Conjunto de datos con igual distribución que $\mathbf{X}$                                     |
| $t$                                        | Dato de entrada con igual distribución que los datos pertenecientes a la clase “normal”       |
| $\hat{t}$                                  | Reconstrucción de $t$ generada por el autocodificador                                         |
| $t^*$                                      | Dato atípico con distinta distribución que $t$                                                |
| $\hat{t}^*$                                | Reconstrucción de $t^*$ generada por el autocodificador                                       |
| $\epsilon$                                 | Umbral calculado a partir del error de reconstrucción de los datos considerados normales      |
| $y^{(u)}$                                  | Etiqueta del ejemplo $x^{(u)}$ , donde 0 indica normal (benigno) y 1 indica anómalo (maligno) |
| $\hat{y}^{(u)}$                            | Predicción de la clase de $x^{(u)}$ por el autocodificador según el umbral $\epsilon$         |



# Resumen

El presente informe aborda la detección de anomalías en datos biomédicos utilizando el conjunto *Breast Cancer Wisconsin (Diagnostic)*, cuyo objetivo es distinguir patrones malignos a partir de muestras benignas obtenidas mediante punción con aguja fina (FNA). Tras un análisis exploratorio inicial, incluyendo distribuciones mediante *boxplots* y la matriz de correlación se implementan autocodificadores entrenados con distintas proporciones de datos de pacientes sanos y enfermos, con el fin de identificar anomalías mediante el error de reconstrucción. Se evalúan tres variantes: el autocodificador clásico, el *sparse autoencoder* y el *contractive autoencoder*, todas desarrolladas en el entorno de *PyTorch*.

Los resultados muestran que, aunque los datos anómalos presentan un mayor error de reconstrucción, los modelos no alcanzaron un desempeño adecuado como clasificadores. Esto indica que la limitación principal no reside en las arquitecturas empleadas, sino en el criterio de decisión para transformar el error de reconstrucción en una etiqueta binaria. En consecuencia, se resalta la necesidad de explorar métodos más robustos y automáticos para la elección del umbral, fundamentales para lograr un sistema funcional en aplicaciones biomédicas.

**explicar umbral**



# Capítulo 1

## Introducción

La detección de anomalías es un campo fundamental dentro del análisis de datos, cuyo objetivo principal es la identificación de patrones que no se ajustan a un comportamiento que coincide con el de la mayoría de los registros. En el contexto de los datos de salud, una anomalía puede representar un error de medición, o una indicación temprana de una condición médica patológica, como el cáncer de mama.

Este trabajo se centra en desarrollar un sistema robusto para la detección automática de tales desviaciones. Se busca resolver la limitación de los métodos estadísticos tradicionales, los cuales a menudo no logran capturar las complejas relaciones no lineales presentes en conjuntos de datos de alta dimensionalidad. El problema específico que se aborda es cómo entrenar un **autocodificador** para que aprenda como identificar si la muestra tomada del área mamaria de un paciente, mediante la aspiración con aguja fina (FNA) se trata de un caso benigno o maligno a partir de un conjunto de datos.

### 1.1. Motivación

El diagnóstico temprano de enfermedades como el cáncer de mama es una tarea prioritaria en la medicina moderna. En muchos casos, los datos obtenidos a través de las muestras contienen patrones sutiles que pueden pasar desapercibidos a través de métodos tradicionales de análisis. Por este motivo, resulta fundamental el desarrollo de herramientas automáticas capaces de detectar irregularidades de manera precisa y rápida.

Las redes neuronales, y en particular los autocodificadores, ofrecen un enfoque adecuado para esta tarea. Un autocodificador es un tipo de red neuronal diseñado para aprender una representación comprimida de los datos, capturando sus características más relevantes. Esta capacidad lo convierte en un buen candidato para identificar si un nuevo registro sigue la misma distribución que los registros normales con los que fue entrenado el modelo.

La incorporación de este tipo de modelos en sistemas de monitoreo podría asistir a profesionales de la salud en la detección temprana de pacientes en riesgo. Por ello, en este proyecto se emplea un autocodificador como herramienta para analizar muestras tomadas de pacientes y explorar su utilidad como detector de posibles casos de cáncer de mama.

### 1.2. Estado del Arte

El uso de técnicas de aprendizaje profundo para la detección de anomalías ha generado un amplio debate académico en torno a la fiabilidad y eficiencia de las arquitecturas basadas en reconstrucción.

Este debate ha impulsado múltiples líneas de investigación que analizan las limitaciones de los autocodificadores tradicionales, comparan diferentes variantes y exploran estrategias alternativas para mejorar la robustez de estos modelos en dominios complejos.

En una crítica central a este supuesto, Bouman y Heskes [2] realizan un análisis teórico y experimental que demuestra que los autocodificadores clásicos son inherentemente poco fiables en escenarios de detección de anomalías. A partir de un estudio que inicia con el análisis de componentes principales (PCA) y autocodificadores lineales, los autores prueban matemáticamente que es posible reconstruir de manera casi perfecta datos anómalos alejados de la distribución de entrenamiento, lo que produce una pérdida de reconstrucción prácticamente nula. Su evidencia experimental, basada en datos tabulares y en el conocido dataset de imágenes MNIST, confirma que este fenómeno persiste también en modelos no lineales. Concluyen que la pérdida de reconstrucción constituye un indicador poco fiable para la identificación de anomalías y que confiar exclusivamente en ella puede conducir a un número significativo de falsos negativos, especialmente en aplicaciones sensibles a errores.

Motivados en parte por estas limitaciones señaladas en los autocodificadores clásicos, Neloy y Turgeon [8] realizan una evaluación comparativa sistemática sobre once arquitecturas aplicadas a la detección de anomalías no supervisada (UAD). Estas arquitecturas, agrupadas en modelos clásicos, variacionales y regularizados, son evaluadas en términos de capacidad de reconstrucción, calidad de generación de muestras e interpretabilidad del espacio latente, empleando experimentos de reproducibilidad sobre los datasets MNIST y Fashion-MNIST junto con métricas como ROC-AUC. Los autores identifican compromisos claros entre fidelidad reconstructiva y estructura latente: mientras los métodos variacionales avanzados (como VQ-VAE y  $\beta$ -VAE) mejoran la organización del espacio latente mediante un control explícito de representaciones discretas, tienden a generar reconstrucciones borrosas. Por el contrario, los modelos deterministas, como el *sparse autoencoder* (SAE) o el *denoising autoencoder* (DAE), muestran mayor precisión reconstructiva y mejor desempeño ante ruido, pero menor robustez ante variaciones complejas de la distribución. Este estudio aporta así una visión integradora sobre las fortalezas y limitaciones prácticas de cada familia de autocodificadores en UAD.

Continuando con esta línea de investigación orientada a superar las limitaciones tradicionales, Walczynska et al. [12] proponen un enfoque centrado en la manipulación directa del espacio latente y en el uso de funciones de costo modificadas. En un análisis dirigido principalmente al tráfico de red, comparan autocodificadores estándar y variacionales, destacando el rol crítico de la estructura latente para discriminar comportamientos anómalos. Los autores introducen un autocodificador variacional con función M-ELBO que incorpora información de anomalías durante el entrenamiento sin intentar reconstruirlas, lo que permite separar de forma más efectiva las regiones latentes correspondientes a comportamiento normal y anómalo. Su marco experimental, ligero y escalable mediante la integración del *stack* ELK y Apache Spark, demuestra beneficios claros en sensibilidad y especificidad, especialmente en flujos de datos de alta intensidad. En conjunto, su trabajo evidencia que el control explícito del espacio latente y el uso de pérdidas modificadas constituyen estrategias efectivas para mejorar la detección en entornos complejos.

**cambiar párrafo entero**

Finalmente, en dominios donde la fiabilidad es crítica, como el biomédico, se han explorado estrategias adicionales fundamentadas en la combinación de modelos. En este contexto, Nawaz et al. [7] presentan una revisión narrativa exhaustiva sobre el uso de ensambles de autocodificadores (EoAE), destacando que la heterogeneidad de los datos médicos incluyendo modalidades como rayos X, MRI, CT, ECG y EEG y la rareza de las anomalías vuelven insuficiente el uso de un único autocodificador. Su revisión muestra

que los ensambles, contruidos mediante la diversificación de hiperparámetros y combinados mediante estrategias como votación o valores promedios, permiten reducir la generalización excesiva observada en modelos individuales y detectar desviaciones sutiles que podrían pasar desapercibidas. No obstante, señalan desafíos importantes, entre ellos la escalabilidad, la falta de interpretabilidad inherente a modelos de tipo “caja negra” y el coste computacional asociado a ejecutar múltiples autocodificadores en paralelo, lo que plantea direcciones claras para investigaciones futuras.



## Capítulo 2

# Materiales y Métodos

En este capítulo se describe el conjunto de datos utilizado, de donde se obtuvo y un análisis exploratorio al mismo, con el propósito de observar las diversas relaciones y distribuciones de las variables.

Por otra parte, el método empleado para la detección de anomalías es el **Autocodificador (Auto-encoder)**. Un autocodificador es un tipo de red neuronal artificial diseñada para aprender una representación codificada de un conjunto de datos. La arquitectura se compone de dos partes principales: el **Codificador (Encoder)** y el **Decodificador (Decoder)**. El funcionamiento general de esta arquitectura puede observarse en la Figura 2.1.

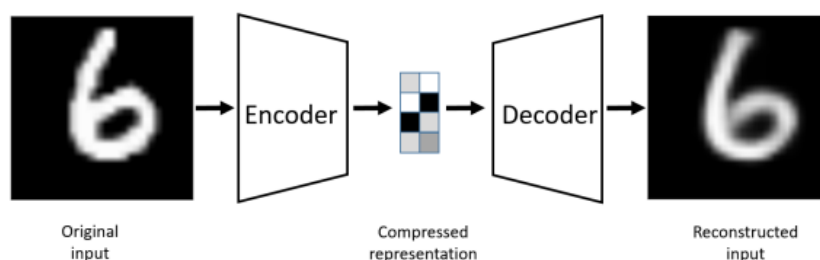


Figura 2.1: Ejemplo de un autocodificador, donde se codifica y decodifica una imagen (tomado de [1]).

### 2.1. Conjunto de datos y AED

Para la realización de este trabajo, se utiliza el conjunto de datos *Breast Cancer Wisconsin (Diagnostic) Data Set*, el cual obtuve de la plataforma Kaggle [6], que es una copia del dataset original publicado por Wolberg et al. [13]. Este conjunto de datos es ampliamente utilizado en tareas de clasificación y análisis de anomalías en el ámbito biomédico. Antes de aplicar cualquier técnica de modelado, se realiza un análisis exploratorio de datos (AED) con el objetivo de comprender la distribución de las características, detectar posibles valores atípicos y observar la relación entre las variables que describen los núcleos celulares.

#### explicar núcleos celulares

El conjunto de datos está compuesto por 569 registros de muestras citológicas y 32 atributos, de los cuales 30 corresponden a características numéricas derivadas del análisis de imágenes digitalizadas, correspondientes a aspiraciones con aguja fina de masas mamarias. Las 2 características restantes son categóricas, *ID*, el cual es solo un metadato que no se utiliza para el entrenamiento del modelo y el **diagnóstico** que puede ser benigno o maligno. Las muestras incluidas en el conjunto de datos fueron

recolectadas entre enero de 1989 y noviembre de 1991.

Las **muestras citológicas** se refieren a grupos celulares obtenidas de las masas mamarias mediante la técnica de aspiración con aguja fina (FNA), que es un procedimiento poco invasivo utilizado para obtener material celular mediante la punción de la masa mamaria con una aguja de pequeño calibre. Esta técnica, ampliamente empleada en el diagnóstico de tumores mamarios, permite recolectar células para su posterior análisis citológico, evitando métodos quirúrgicos más invasivos. Su uso en el estudio del cáncer de mama se remonta a la década de 1930 y ha demostrado poseer una elevada precisión diagnóstica cuando es ejecutada e interpretada por un patólogo especializado.

La FNA posibilita examinar características celulares y arquitectónicas del tejido aspirado, así como obtener material apto para tinciones citológicas que contribuyen a la detección de alteraciones malignas. Debido a su eficacia, simplicidad y bajo riesgo, la técnica se ha consolidado como una herramienta fundamental en la evaluación citológica de lesiones mamarias [4].

Las diez características base del conjunto de datos fueron obtenidas mediante las metodologías computacionales detalladas en el artículo original de Street et al. [11]. Estas características describen propiedades estructurales de los núcleos celulares, tales como tamaño, forma, textura y simetría, y se muestran en la Tabla 2.1.

Tabla 2.1: Descripción de las diez características base extraídas de las imágenes digitalizadas de FNA.

| No. | Característica                                        | Descripción                                                    |
|-----|-------------------------------------------------------|----------------------------------------------------------------|
| 1   | <b>Radio</b> ( <i>radius</i> )                        | Media de las distancias del centro a los puntos del perímetro. |
| 2   | <b>Textura</b> ( <i>texture</i> )                     | Desviación estándar de los valores de la escala de grises.     |
| 3   | <b>Perímetro</b> ( <i>perimeter</i> )                 | Medida del contorno del núcleo.                                |
| 4   | <b>Área</b> ( <i>area</i> )                           | Superficie cubierta por el núcleo.                             |
| 5   | <b>Suavidad</b> ( <i>smoothness</i> )                 | Variación local en las longitudes del radio.                   |
| 6   | <b>Compacidad</b> ( <i>compactness</i> )              | $\text{Perímetro}^2 / \text{área} - 1,0$ .                     |
| 7   | <b>Concavidad</b> ( <i>concavity</i> )                | Severidad de las porciones cóncavas del contorno.              |
| 8   | <b>Puntos Cóncavos</b> ( <i>concave points</i> )      | Número de porciones cóncavas del contorno.                     |
| 9   | <b>Simetría</b> ( <i>symmetry</i> )                   | Nivel de simetría del núcleo celular.                          |
| 10  | <b>Dimensión Fractal</b> ( <i>fractal dimension</i> ) | Aproximación a la “línea de costa” menos 1.                    |

para cada una de estas diez características se calculan tres medidas distintas, lo que completa los 30 atributos finales del conjunto de datos:

1. La **media** de la característica.
2. El **error estándar** (*Standard Error*, SE) de la característica.
3. El **'peor'** valor o más grande, el cual se calcula como la media de los tres valores más grandes de la característica.

Iniciando el proceso de **análisis exploratorio de datos (AED)**, se verifica la integridad del conjunto. Tras la revisión de todos los registros y atributos, se confirma la ausencia total de datos faltantes, lo que hizo innecesaria cualquier etapa de imputación o limpieza de datos previa.

Tras el análisis general de los atributos, se examina la **distribución de la variable objetivo**, *diagnosis*, con el fin de comprender el balance entre las clases presentes en el conjunto de datos. La distribución se muestra en la Tabla 2.2.



Tabla 2.2: Distribución de la variable objetivo *diagnosis*

| Valor       | Porcentaje (%) | Cantidad de registros |
|-------------|----------------|-----------------------|
| B (Benigno) | 62.74          | 357                   |
| M (Maligno) | 37.26          | 212                   |

esta distribución revela un leve desbalance entre ambas categorías, algo esperable para este trabajo, donde se espera detectar una anomalía, es decir, un diagnóstico maligno.

Para examinar la **distribución de las características** en función de la naturaleza del registro (normal o anómalo), se genera un gráfico de caja múltiple (*boxplot*) para las variables seleccionadas. Esta visualización, mostrada en la Figura 2.2, es crucial para identificar diferencias en la dispersión y tendencia central, así como la presencia de **valores atípicos** en cada grupo. Al comparar las distribuciones, se observa que el *Area Mean* (área promedio) presenta la distinción más clara entre los datos normales y anómalos. En contraste, el *Fractal Dimension Mean* (dimensión fractal promedio) muestra una superposición significativa entre las categorías, y las diferencias son más sutiles para el *Symmetry Mean* (simetría promedio).

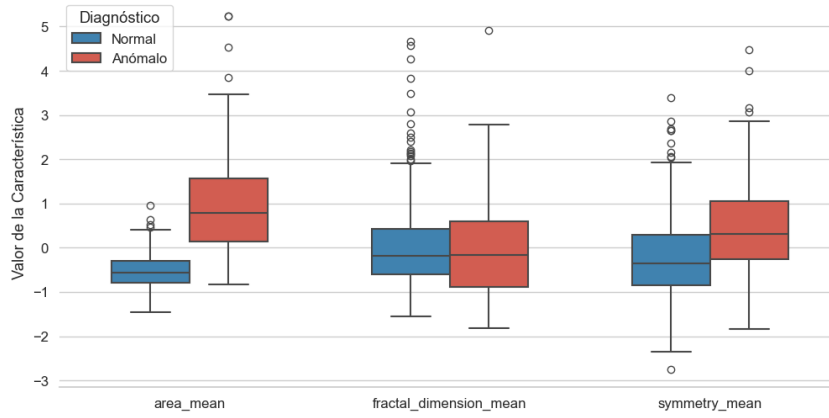


Figura 2.2: Boxplot combinado de *Area Mean*, *Fractal Dimension Mean* y *Symmetry Mean*, segmentado por (normal/anómalo).

Una vez analizada la distribución de las variables individualmente, se procede a calcular la matriz de correlación  $\Sigma$ , la cual mide el grado de relación entre cada par de atributos del conjunto de datos. Este valor está acotado entre -1 y 1, donde 1 indica una relación lineal perfecta, -1 una relación lineal inversa perfecta y 0 denota que no existe una relación lineal entre los atributos. La matriz de correlación se define en la Ecuación 2.1 para cada par de atributos  $(X_i, X_j) \in \mathbf{X}$ .

$$\Sigma = \begin{pmatrix} \text{Corr}(X_1, X_1) & \text{Corr}(X_1, X_2) & \cdots & \text{Corr}(X_1, X_d) \\ \text{Corr}(X_2, X_1) & \text{Corr}(X_2, X_2) & \cdots & \text{Corr}(X_2, X_d) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Corr}(X_d, X_1) & \text{Corr}(X_d, X_2) & \cdots & \text{Corr}(X_d, X_d) \end{pmatrix} \in \mathbb{R}^{d \times d} \quad (2.1)$$

donde la correlación entre dos atributos  $\text{Corr}(X_1, X_2)$  se muestra en la Ecuación 2.2.

$$\text{Corr}(X_i, X_j) = \frac{\text{Cov}(X_i, X_j)}{s_{x_i} s_{x_j}} \quad (2.2)$$

y  $\text{Cov}(X_i, X_j)$  representa la covarianza entre  $X_i$  y  $X_j$ , mientras que  $s_{x_i}$  y  $s_{x_j}$  son sus desviaciones estándar. Finalmente, la covarianza utilizada en ese cálculo se expresa en la Ecuación 2.3, que mide la existencia

de asociación lineal entre los dos atributos.

$$\text{Cov}(X_i, X_j) = \frac{1}{n} \sum_{t=1}^n (x_{ti} - \bar{x}_i)(x_{tj} - \bar{x}_j) \quad (2.3)$$

donde  $\bar{x}_i$  es la media muestral del atributo  $X_i$ , definida como:

$$\bar{x}_i = \frac{1}{n} \sum_{t=1}^n x_{ti} \quad (2.4)$$

y  $s_{x_i}$  es el desvío estándar muestral asociado a  $X_i$ , dado por:

$$s_{x_i} = \sqrt{\frac{1}{n-1} \sum_{t=1}^n (x_{ti} - \bar{x}_i)^2} \quad (2.5)$$

La matriz de correlación correspondiente a las características del conjunto de datos de este trabajo se presenta en la Figura 2.3, donde pueden observarse las relaciones lineales entre los distintos atributos. En la misma se observa una fuerte relación entre el área, el perímetro y el radio, algo esperable teniendo en cuenta las relaciones entre estas medidas.

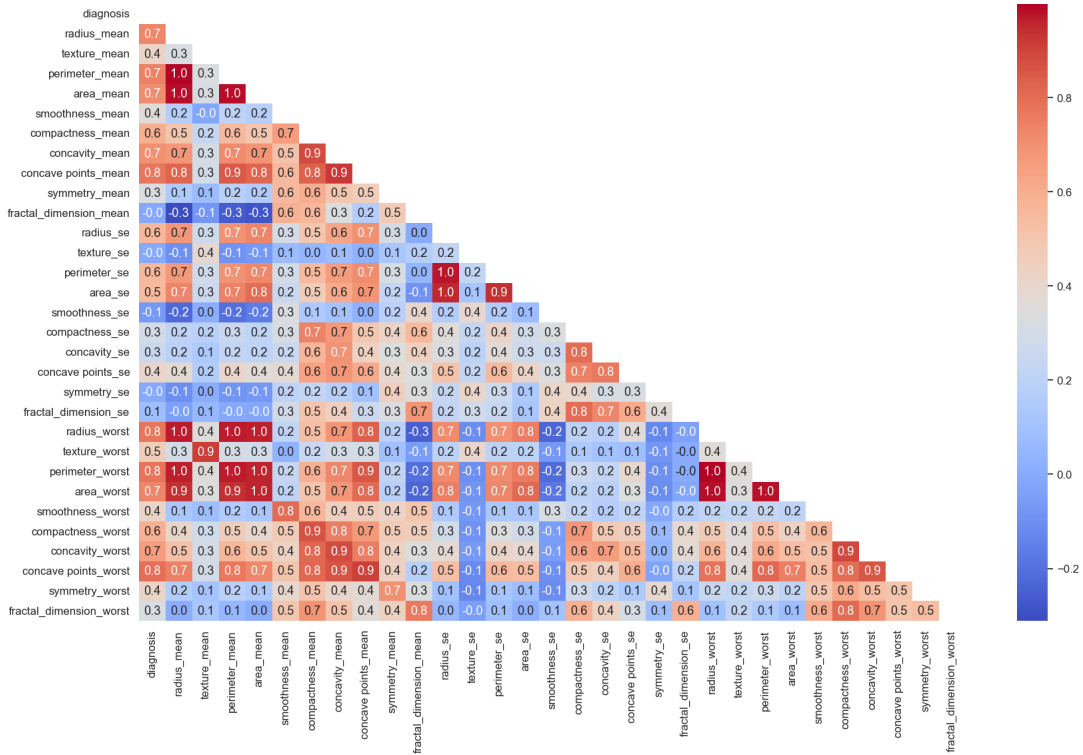


Figura 2.3: Matriz de correlación entre los atributos del conjunto de datos.

### comentar sobre relaciones negativas

Para analizar la robustez del autocodificador frente a distintos niveles de contaminación en los datos, se realiza una división inicial del conjunto de datos en tres subconjuntos: **entrenamiento**, **validación** y **prueba**, manteniendo en todos la proporción original de clases, tal como se indica en el paso (1) de la Figura 2.4. En esta división, el 60 % de las muestras se destinó al entrenamiento, el 20 % al conjunto de validación y el 20 % restante al conjunto de prueba, tal como se muestra en la Tabla 2.3.

Tabla 2.3: División del conjunto original en entrenamiento, validación y prueba, manteniendo la proporción original de clases.

| Subconjunto   | Proporción | Total | Normales | Anómalos |
|---------------|------------|-------|----------|----------|
| Entrenamiento | 60 %       | 341   | 214      | 128      |
| Validación    | 20 %       | 114   | 72       | 42       |
| Prueba        | 20 %       | 114   | 72       | 42       |

El subconjunto de validación se utiliza con dos propósitos principales: evitar el sobreajuste (*overfitting*) del modelo durante el entrenamiento empleando únicamente los registros **normales** para este control y calcular las métricas necesarias durante el proceso de *grid search* para la selección de hiperparámetros.

Luego, se aplica una estandarización a todos los atributos de cada subconjunto de manera independiente. Este método transforma las variables para que tengan una media ( $\mu$ ) de cero y una desviación estándar ( $\sigma$ ) de uno, asegurando que todas las entradas del modelo tengan una escala comparable. Esta estandarización también contribuye a mitigar posibles problemas de *overflow* en las activaciones del autocodificador, los cuales pueden surgir cuando el modelo recibe valores excesivamente grandes. Este proceso se define formalmente en la Ecuación 2.6.

$$z = \frac{x - \mu}{\sigma} \quad (2.6)$$

donde  $x$  corresponde al dato original y tanto  $\mu$  como  $\sigma$ , a la media y la desviación estándar de esa variable, respectivamente.

La estandarización de los tres subconjuntos (entrenamiento, validación y prueba) se realiza utilizando exclusivamente la media y el desvío estándar calculados a partir del **subconjunto de entrenamiento**. Esta estrategia evita la fuga de información o *data leakage*, fenómeno que ocurre cuando se incorporan, directa o indirectamente, estadísticas derivadas de los datos de validación o prueba durante el entrenamiento del modelo. La fuga de información genera una visión artificialmente optimista del desempeño, ya que introduce en el proceso señales que el modelo no debería conocer antes de ser evaluado.

A partir del subconjunto de entrenamiento, se generan tres nuevos subconjuntos con diferentes proporciones de anomalías, proceso señalado como paso (2) en la Figura 2.4. El objetivo es evaluar cómo afecta al modelo entrenar con distintos niveles de contaminación, manteniendo constante el tamaño total de cada conjunto y preservando la separación previa entre entrenamiento, validación y prueba establecida en el esquema general mostrado en la figura.

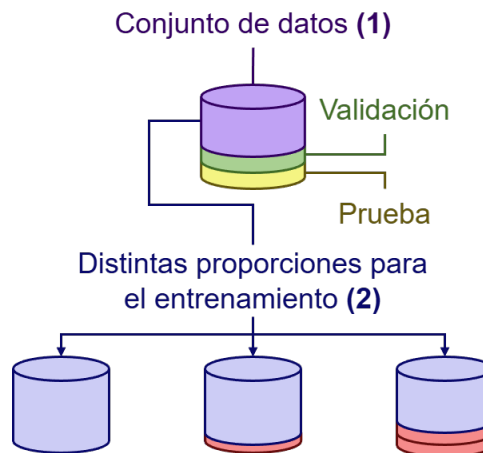


Figura 2.4: Esquema de la división del conjunto de datos.

La Tabla 2.4 resume las proporciones definidas para cada subconjunto derivado del entrenamiento. El subconjunto A contiene únicamente registros normales, mientras que B y C incorporan niveles crecientes de anomalías (10 % y 25 %, respectivamente).

Tabla 2.4: Subconjuntos derivados del subconjunto de entrenamiento para evaluar la resistencia del autocodificador ante diferentes proporciones de anomalías.

| ID | Proporción Anómalos | Proporción Normales | Registros Anómalos | Registros Normales |
|----|---------------------|---------------------|--------------------|--------------------|
| A  | 0 %                 | 100 %               | 0                  | 214                |
| B  | 10 %                | 90 %                | 21                 | 193                |
| C  | 25 %                | 75 %                | 53                 | 161                |

## 2.2. Descripción de los métodos

En esta sección se presentan los conceptos centrales utilizados en el desarrollo del trabajo. Primero, se describe formalmente la arquitectura del **autocodificador** clásico y dos de sus variantes: *sparse auto-encoder* y *contractive autoencoder*. Luego, el modo en que pueden emplearse como detector de anomalías, mediante el análisis del error de reconstrucción y estableciendo un criterio de decisión.

Con el criterio de decisión definido, se detallan las **métricas de evaluación** empleadas para cuantificar el desempeño del modelo, incluyendo las medidas estándar derivadas de la matriz de confusión, así como herramientas adicionales de análisis como la **curva ROC** y el **AUC**, que permiten evaluar el comportamiento del detector bajo diferentes umbrales de decisión.

### 2.2.1. Autocodificadores

El método empleado en este trabajo es el **autocodificador**, un tipo de perceptrón multicapa caracterizado por su estructura simétrica respecto a la capa latente y por poseer la misma dimensión tanto en la entrada como en la salida. Su objetivo es aprender una representación comprimida que permita reconstruir los datos originales.

#### pasar a estado del arte

Los autocodificadores surgen de las primeras investigaciones en redes neuronales orientadas a aprender representaciones internas útiles sin supervisión explícita. En particular, el trabajo de Rumelhart,

Hinton y Williams [5] introdujo el uso del algoritmo de retropropagación para aprender representaciones internas, sentando las bases conceptuales para modelos capaces de reconstruir sus propias entradas. Posteriormente, el enfoque fue formalizado explícitamente por Bourlard y Kamp [3], quienes describieron la autoasociación mediante perceptrones multicapa y demostraron su relación con métodos clásicos de reducción de dimensionalidad como la descomposición en valores singulares (SVD). Estos desarrollos consolidaron al autocodificador como un modelo central para el aprendizaje de representaciones de baja dimensión.

La arquitectura de un autocodificador se compone de dos partes principales: un **codificador**, que transforma los datos de entrada en una representación de menor dimensionalidad, y un **decodificador**, que intenta reconstruir la entrada original a partir de dicha representación compacta. Entre ambos se encuentra la **capa latente**, donde se almacena la representación interna de menor dimensión. La Figura 2.5 ilustra la arquitectura general empleada en este trabajo, mostrando el flujo desde la entrada  $X$ , su codificación en el espacio latente  $Z$ , y la posterior reconstrucción  $\hat{X}$  realizada por el decodificador.

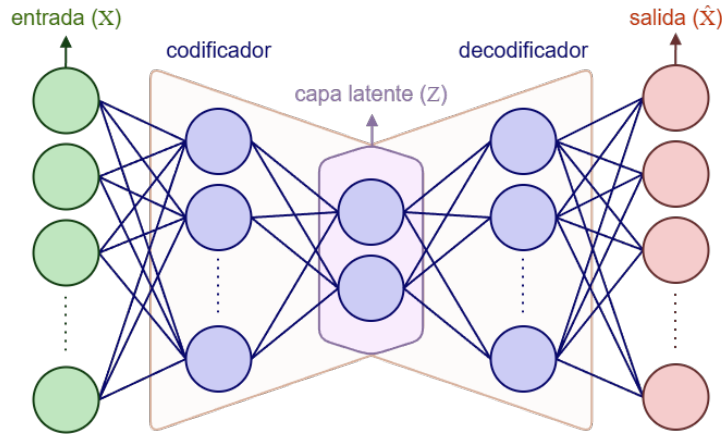


Figura 2.5: Arquitectura general de un autocodificador.

Para formalizar este proceso, considérese un conjunto de entrenamiento:

$$\mathbf{X} = \{x^{(1)}, x^{(2)}, \dots, x^{(N)}\}, \quad x^{(u)} \in \mathbb{R}^D, \quad u \in 1 \dots N, \quad (2.7)$$

donde cada ejemplo puede escribirse como:

$$x^{(u)} = (x_1^{(u)}, x_2^{(u)}, \dots, x_D^{(u)}), \quad u \in 1 \dots N, \quad (2.8)$$

y el conjunto de salidas deseadas se define como:

$$\hat{x}^{(u)} = x^{(u)}, \quad (2.9)$$

donde  $\hat{x}^{(u)}$  es la salida generada por el modelo y la salida esperada es la propia entrada  $x^{(u)}$ . Lo que implica que, aunque típicamente se lo describe como un modelo no supervisado por no requerir etiquetas externas, el aprendizaje es **supervisado**, pues la red debe aprender a reproducir su propia entrada.

En este esquema, el **codificador** proyecta el vector de entrada  $x^{(u)}$  a una representación comprimida  $z^{(u)} \in \mathbb{R}^K$ , donde  $K < D$ , a través de una transformación no lineal:

$$z^{(u)} = \text{codificador}(x^{(u)}), \quad (2.10)$$

mientras que el **decodificador** intenta reconstruir la entrada original generando:

$$\hat{x}^{(u)} = y^{(u)} = \text{decodificador}(z^{(u)}), \quad (2.11)$$

dando lugar al proceso completo de codificación y decodificación.

El entrenamiento del autocodificador se realiza mediante retropropagación, minimizando una función de costo  $\mathcal{L}$  que mide la diferencia entre cada entrada  $x^{(u)}$  y su reconstrucción  $\hat{x}^{(u)}$ . Una forma clásica de esta función está dada por:

$$\mathcal{L} = \frac{1}{ND} \sum_{u=1}^N \sum_{k=1}^D \left( x_k^{(u)} - \hat{x}_k^{(u)} \right)^2, \quad (2.12)$$

equivalente al error cuadrático medio (MSE por sus siglas en inglés) sobre todas las muestras, donde  $N$  es la cantidad de registros y  $D$  es la dimensión de  $x$ . Este error descrito en la Ecuación 2.12 guía el ajuste de los parámetros del codificador y del decodificador para mejorar la calidad de reconstrucción.

La **no linealidad** presente en el autocodificador surge del uso de funciones de activación aplicadas en cada neurona del modelo. En esencia, cada unidad realiza la operación mostrada en la Ecuación 2.13.

$$a_j^{(u)} = g \left( \sum_{k=1}^D w_{jk} x_k^{(u)} \right), \quad (2.13)$$

donde  $x_k^{(u)}$  representa el valor de la  $k$ -ésima componente del vector de entrada  $x^{(u)}$  que ingresa a la neurona  $j$ ,  $w_{jk}$  es el peso sináptico que regula la contribución de dicha entrada en la activación y  $a_j^{(u)}$  la activación obtenida en dicha neurona. La función  $g(x)$  es una función de activación no lineal por ejemplo,  $\tanh(\beta x)$  o funciones sigmoideas relacionadas que permiten que la red modele relaciones complejas que no podrían representarse mediante transformaciones puramente lineales. Este componente no lineal es esencial para que el autocodificador pueda aprender estructuras internas del conjunto de datos y generar representaciones latentes útiles.

En este trabajo, la función de activación utilizada en todas las capas ocultas es GELU, debido a sus ventajas frente a la comúnmente empleada ReLU. Entre estas ventajas se encuentran: una superficie de pérdida más suave que facilita un entrenamiento más estable, la ausencia del problema de “neuronas muertas” asociado a ReLU y una mayor capacidad para capturar relaciones no lineales gracias a su formulación probabilística basada en la distribución normal.

La función  $\text{GELU}(x)$  está definida por:

$$\text{GELU}(x) = 0,5 x \left( 1 + \tanh \left( \sqrt{\frac{2}{\pi}} (x + 0,044715x^3) \right) \right) \quad (2.14)$$

La función  $\text{ReLU}(x)$  se define como:

$$\text{ReLU}(x) = \max(0, x) \quad (2.15)$$

Como se muestra en la Figura 2.6, al graficar ambas funciones se aprecia la suavidad de GELU al aproximarse a 0, en contraste con el comportamiento abrupto de ReLU, lo que evita discontinuidades y favorece un entrenamiento más estable en modelos neuronales.

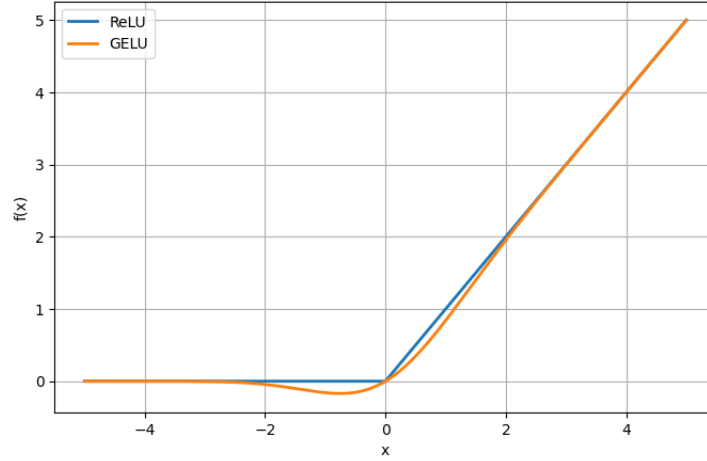


Figura 2.6: Funciones de activación ReLU y GELU.

Si bien el autocodificador estándar es efectivo para reducir la dimensionalidad, asegurando que la capa latente sea de menor dimensión ( $K < D$ ), existen variantes que imponen restricciones adicionales sobre la representación latente  $\mathbf{z}^{(u)} \in \mathbb{R}^K$  para evitar que el modelo simplemente aprenda la función identidad y para forzarlo a capturar características más robustas y útiles de los datos.

El **Autocodificador Disperso (SAE)** [9] impone una restricción de **dispersión** (sparsity) sobre la capa latente. En este caso, el objetivo no es necesariamente reducir la dimensión ( $K$  puede ser menor o incluso mayor que  $D$ ), sino asegurar que la mayoría de las neuronas en la capa latente permanezcan **inactivas** (con activación cercana a cero) para cada ejemplo  $x^{(u)}$ . Esta dispersión fuerza al modelo a utilizar diferentes subconjuntos de neuronas para representar distintas entradas, lo que previene el sobreajuste y favorece la extracción de características más discriminativas.

La función de costo total del SAE se denota como  $\mathcal{L}_{\text{sparse}}(W, b)$  y está compuesta por el término de reconstrucción  $\mathcal{L}(W, b)$  más un término de penalización de dispersión:

$$\mathcal{L}_{\text{sparse}}(W, b) = \mathcal{L}(W, b) + \beta \sum_{j=1}^H \text{KL}(\rho \parallel \hat{\rho}_j), \quad (2.16)$$

### explicar H

donde  $W$  denota el conjunto de **matrices de pesos** de todas las capas del autocodificador (tanto del codificador como del decodificador),  $b$  corresponde al conjunto de **vectores de sesgo** o *bias* asociados a dichas capas y  $\beta$  es el hiperparámetro que controla el peso de la penalización. El término de penalización es la suma de la **divergencia Kullback-Leibler** (KL) entre la tasa de activación promedio deseada ( $\rho$ , un valor pequeño fijo (0.05 en el artículo original) y la tasa de activación promedio real ( $\hat{\rho}_j$ ) de cada neurona  $j$  en la capa oculta:

$$\sum_{j=1}^H \text{KL}(\rho \parallel \hat{\rho}_j) = \sum_{j=1}^H \left[ \rho \log \left( \frac{\rho}{\hat{\rho}_j} \right) + (1 - \rho) \log \left( \frac{1 - \rho}{1 - \hat{\rho}_j} \right) \right] \quad (2.17)$$

### cambiar explicación $\rho$

Esta penalización tiene la propiedad de ser cero si  $\hat{\rho}_j = \rho$ , y aumentar a medida que  $\hat{\rho}_j$  se aleja de  $\rho$ . Al minimizar esta función, el SAE es obligado a tener representaciones latentes dispersas.

Otra variante interesante es el **Autocodificador Contractivo (CAE)**[10], este se enfoca en hacer que la representación latente  $\mathbf{z}^{(u)}$  sea **robusta** frente a pequeñas variaciones o ruido en los datos de

entrada  $x^{(u)}$ . El objetivo es que la función de codificación  $f$  sea **contractiva**, es decir, que un pequeño cambio en el vector de entrada produzca un cambio muy pequeño en la representación latente. Esto obliga al modelo a capturar únicamente los **factores de variación más relevantes**, ignorando perturbaciones irrelevantes.

El CAE introduce un término de penalización basado en la sensibilidad del codificador. Esta sensibilidad está dada por la **matriz Jacobiana** del codificador evaluada en cada ejemplo  $x^{(u)}$ :

$$J_f(x^{(u)}) = \frac{\partial f(x^{(u)})}{\partial x^{(u)}} \in \mathbb{R}^{K \times D}. \quad (2.18)$$

La penalización se define como el **cuadrado de la norma de Frobenius** del Jacobiano. Para una matriz  $A \in \mathbb{R}^{m \times n}$ , la norma de Frobenius se define como:

$$\|A\|_F^2 = \sum_{i=1}^m \sum_{j=1}^n A_{ij}^2. \quad (2.19)$$

Con esta definición, la función de costo total del CAE es:

$$\mathcal{L}_{\text{CAE}}(W, b) = \mathcal{L}(W, b) + \lambda \sum_{u=1}^N \left\| J_f(x^{(u)}) \right\|_F^2, \quad (2.20)$$

donde  $\mathcal{L}(W, b)$  es el error de reconstrucción (por ejemplo, MSE) y  $\lambda > 0$  controla el peso del término contractivo.

Al minimizar  $\mathcal{L}_{\text{CAE}}(W, b)$ , el codificador aprende un mapeo  $x^{(u)} \mapsto Z^{(u)}$  que es **suave, estable y resistente al ruido**, haciéndose insensible a variaciones irrelevantes y destacando las características realmente importantes.

A partir de estas arquitecturas, el autocodificador y sus variantes pueden emplearse como un método de **detección de anomalías**. Para ello, se deben tener en cuenta las etiquetas de cada ejemplo del conjunto de datos, definidas en la Ecuación 2.21:

$$y^{(u)} = \begin{cases} 0, & \text{representa un caso benigno (normal)} \\ 1, & \text{representa un caso maligno (anomalía)} \end{cases} \quad (2.21)$$

Con esta definición, el modelo se entrena exclusivamente utilizando el subconjunto de ejemplos que representan el comportamiento “normal”, tal como se especifica en la Ecuación 2.22:

$$\mathbf{X}_{\text{benigno}} = \{ x^{(u)} \in \mathbf{X} \mid y^{(u)} = 0 \}. \quad (2.22)$$

es decir, masas tumorales benignas. Al aprender a reconstruir correctamente este tipo de patrones, cuando el modelo recibe un nuevo dato  $t$  perteneciente a un conjunto  $\mathbf{T}$  con la misma distribución que  $\mathbf{X}$ , se espera que el error de reconstrucción  $\|\hat{t} - t\|$  sea pequeño.

En contraste, si se presenta un dato atípico  $t^*$ , correspondiente a un diagnóstico maligno, el modelo no logra reconstruirlo con la misma fidelidad, generando así un error significativamente mayor. Este comportamiento permite establecer un criterio simple y efectivo para la detección de anomalías: un dato se clasifica como anómalo si su error de reconstrucción supera un umbral  $\epsilon$  predefinido. El criterio se expresa formalmente en la Ecuación 2.23:

$$\|\hat{t}^* - t^*\| > \epsilon \quad (2.23)$$



donde  $\hat{t}^*$  es la reconstrucción producida por la red a partir del dato  $t^*$ . Si la desigualdad se cumple, el dato se considera **anómalo**; de lo contrario, se clasifica como **normal**.

El valor del umbral  $\epsilon$  se determinó de manera empírica a partir del conjunto de datos normales utilizado durante el entrenamiento. En particular, se calculó como la media del error de reconstrucción más dos veces su desvío estándar. Esta definición se detalla en la Ecuación 2.24:

$$\epsilon = \text{media}(\|\hat{x}^{(u)} - x^{(u)}\|) + 2 \cdot \text{std}(\|\hat{x}^{(u)} - x^{(u)}\|), \quad (2.24)$$

donde  $x^{(u)}$  corresponde a cada muestra normal del conjunto de entrenamiento  $\mathbf{X}_{\text{benigno}}$  y  $\hat{x}^{(u)}$  a su respectiva reconstrucción generada por el modelo. Este valor proporciona un límite representativo del comportamiento esperado en datos no anómalos, permitiendo distinguir de forma efectiva aquellos casos cuyo error excede significativamente lo aprendido por la red.

### 2.2.2. Métricas de evaluación del modelo

Para evaluar rigurosamente el desempeño del autocodificador en la tarea de detección de anomalías, se analizan diversas métricas derivadas del error de reconstrucción  $\|\hat{t} - t\|$ . Dado que este valor actúa como cuantificador directo de qué tan bien el modelo reproduce cada instancia, es posible utilizarlo como criterio de decisión para distinguir entre casos normales y anómalos mediante un umbral  $\epsilon$ .

A partir de esta regla se construye la matriz de confusión, desde la cual se obtienen métricas clásicas de evaluación como la exactitud, la sensibilidad y la especificidad. Asimismo, se emplean herramientas adicionales basadas en la variación del umbral, como la curva ROC y el área bajo dicha curva (AUC) que permiten caracterizar el rendimiento global del modelo sin depender de un valor fijo de  $\epsilon$ , proporcionando una visión más completa de su capacidad discriminativa en términos del error de reconstrucción.

Para obtener un criterio de decisión utilizamos el umbral  $\epsilon$  definido en la Ecuación 2.24, a partir del cual, cada instancia se clasifica como **normal** o **anómala** según si su error de reconstrucción  $\|\hat{t} - t\|$  supera o no el umbral  $\epsilon$ , tal como se muestra en la Ecuación 2.25.

$$\hat{y} = \begin{cases} 0, & \text{si } \|\hat{t} - t\| \leq \epsilon \\ 1, & \text{si } \|\hat{t} - t\| > \epsilon \end{cases} \quad (2.25)$$

A partir de este criterio, se distinguen los siguientes cuatro casos fundamentales:

- **Verdaderos Positivos (TP)**: Casos *anómalos* (malignos) cuyo error de reconstrucción satisface  $\|\hat{t} - t\| > \epsilon$ .
- **Falsos Negativos (FN)**: Casos *anómalos* (malignos) cuyo error cumple  $\|\hat{t} - t\| \leq \epsilon$ , es decir, el modelo no detectó la anomalía.
- **Verdaderos Negativos (TN)**: Casos *normales* (benignos) cuyo error satisface  $\|\hat{t} - t\| \leq \epsilon$ .
- **Falsos Positivos (FP)**: Casos *normales* (benignos) cuyo error satisface  $\|\hat{t} - t\| > \epsilon$ , siendo incorrectamente clasificados como anómalos.

A partir de estas cuatro categorías se construye la matriz de confusión mostrada en la Tabla 2.5, donde la diagonal verde representa las predicciones correctas y las celdas rojas los errores de clasificación.

Tabla 2.5: Matriz de confusión. La diagonal verde representa predicciones correctas, mientras que las celdas rojas corresponden a errores de clasificación.

|      |         | Predicción              |                         |
|------|---------|-------------------------|-------------------------|
|      |         | Anómalo                 | Normal                  |
| Real | Anómalo | Verdadero Positivo (TP) | Falso Negativo (FN)     |
|      | Normal  | Falso Positivo (FP)     | Verdadero Negativo (TN) |

Una vez definidas las categorías de la matriz de confusión, es posible cuantificar el rendimiento del modelo mediante distintas métricas, cada una evaluando un aspecto particular de la calidad de las predicciones. Todas ellas se derivan directamente de los valores TP, FP, TN y FN.

La **exactitud** (*Accuracy*) cuantifica la proporción total de predicciones correctas realizadas por el modelo según la Ecuación 2.26.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}} \quad (2.26)$$

La precisión (*Precision*) indica qué proporción de las predicciones positivas corresponde realmente a casos positivos, tal como se define en la Ecuación 2.27.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2.27)$$

La sensibilidad (*Recall*) mide la capacidad del modelo para identificar correctamente los casos positivos presentes en el conjunto de datos, de acuerdo con la Ecuación 2.28.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (2.28)$$

La especificidad (*Specificity*) evalúa la proporción de casos negativos correctamente clasificados, definida en la Ecuación 2.29.

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}} \quad (2.29)$$

La exactitud balanceada (*Balanced Accuracy*), mostrada en la Ecuación 2.30, combina sensibilidad y especificidad para obtener una medida equilibrada del rendimiento.

$$\text{Balanced Accuracy} = \frac{\text{Recall} + \text{Specificity}}{2} \quad (2.30)$$

El *F1-Score* combina las medidas de *Precision* y *Recall* para ofrecer una métrica más general de la calidad del modelo, tal como se expresa en la Ecuación 2.31.

$$\text{F1}_{\text{score}} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \text{ TP}}{2 \text{ TP} + \text{FP} + \text{FN}} \quad (2.31)$$

En el caso particular de este trabajo, resulta crítico priorizar la métrica de *Recall*. Dado que un falso negativo implica no identificar un caso potencialmente maligno. Maximizar esta métrica es esencial para garantizar que el modelo no pase por alto ninguna anomalía.

Si bien *Precision* también aporta información valiosa al reducir el número de falsos positivos, la severidad clínica de un falso negativo vuelve al *Recall* la métrica de mayor importancia en este contexto. No obstante, para obtener una visión más equilibrada del rendimiento global del sistema, también se considera el  $F1_{score}$ , que combina *Recall* y *Precision* en una única medida. De esta manera, aunque el objetivo principal es maximizar el *Recall*, un  $F1_{score}$  elevado asegura que dicho incremento no se logra a costa de elevar los falsos positivos, proporcionando así una evaluación más completa de la calidad del modelo.

Las métricas presentadas hasta ahora (*Accuracy*, *Precision*, *Recall*, *Specificity*, etc.) son dependientes del umbral de decisión  $\epsilon$  elegido. Para obtener una evaluación del rendimiento que sea independiente de la elección de un umbral específico y que caracterice la capacidad discriminativa del modelo en todo el rango de posibles valores del error de reconstrucción, se utiliza la **Curva ROC** y el área bajo esta curva (**AUC**).

La **Curva ROC** es una representación gráfica que resume el comportamiento del clasificador al comparar la proporción de verdaderos positivos contra la proporción de falsos positivos para distintos valores de umbral  $\epsilon$ . Para construirla, se utiliza en el eje vertical *y* la **Tasa de Verdaderos Positivos** (o por sus siglas en inglés TPR de *True Positive Rate*), una medida que cuantifica qué tan eficaz es el modelo para detectar correctamente las anomalías presentes en los datos. Esta tasa se define en la Ecuación 2.32.

$$TPR = \frac{TP}{TP + FN} = \frac{TP}{P} \quad (2.32)$$

De forma análoga, el eje horizontal *x* de la curva utiliza la **Tasa de Falsos Positivos** (o por sus siglas en inglés FPR de *False Positive Rate*), que indica cuántos casos normales son clasificados erróneamente como anómalos. Se describe en la Ecuación 2.33 y esta cantidad es complementaria a la especificidad definida anteriormente en la Ecuación 2.29.

$$FPR = \frac{FP}{FP + TN} = \frac{FP}{N} = 1 - \text{Specificity} \quad (2.33)$$

La curva se genera variando el umbral de decisión  $\epsilon$  a través de todos los valores posibles del error de reconstrucción. Cada valor de  $\epsilon$  resulta en un punto diferente (FPR, TPR) en el espacio ROC. Un clasificador ideal se encuentra en el punto (0, 1) del espacio ROC, lo que corresponde a  $TPR = 1$  (todos los anómalos detectados) y  $FPR = 0$  (ningún normal clasificado erróneamente).

El **Área Bajo la Curva ROC (AUC)** es una métrica escalar que resume el rendimiento global del modelo en una única medida. Se define como el área total delimitada por la curva ROC y los ejes.

El AUC proporciona una medida robusta e independiente del umbral de la capacidad de clasificación del modelo, lo que lo convierte en una métrica fundamental para evaluar el desempeño general. Especialmente en tareas de detección de anomalías, donde la distribución de clases suele ser altamente desbalanceada. Un clasificador es razonable si  $0,5 < AUC \leq 1$ .



## Capítulo 3

# Resultados

En este capítulo se presentan los resultados obtenidos durante el entrenamiento de los autocodificadores propuestos. Para ello, se muestran las configuraciones utilizadas, los histogramas de errores de reconstrucción y finalmente las métricas de desempeño para los distintos modelos y sus variantes entrenadas con los conjuntos de diferentes proporciones, que resumen el rendimiento de cada modelo bajo el criterio de decisión.

Para cada una de las arquitecturas consideradas autocodificador clásico (AE), autocodificador disperso (SAE) y autocodificador contractivo (CAE) se entrenan tres variantes (A, B y C), que comparten exactamente la misma configuración e hiperparámetros y difieren únicamente en el conjunto de entrenamiento utilizado.

Las configuraciones finales empleadas para cada arquitectura surgen de un proceso de *grid search*, en el cual se exploraron distintas combinaciones de cantidad de capas, de neuronas y tasas de aprendizaje (*lr* por sus siglas en inglés de *Learning Rate*). La selección de los hiperparámetros óptimos se basó en el desempeño sobre el conjunto de **validación**, priorizando las métricas de *Recall*, *F1<sub>score</sub>* y AUC. Posteriormente, los modelos seleccionados fueron entrenados con los hiperparámetros con mejores métricas y fueron evaluados sobre el conjunto de prueba, cuyos resultados son los expuestos en este capítulo. La Tabla 3.1 resume los hiperparámetros utilizados para cada una de las arquitecturas finales:

Tabla 3.1: Hiperparámetros de entrenamiento para cada modelo.

| Modelo | Dimensiones de capas           | <i>lr</i> | <i>Batch size</i> | Épocas |
|--------|--------------------------------|-----------|-------------------|--------|
| AE     | [30, 128, 64, 32, 16, 8, 4, 2] | 0.0001    | 32                | 1000   |
| SAE    | [30, 128, 64, 32, 16]          | 0.0001    | 32                | 1000   |
| CAE    | [30, 32, 16, 8, 4]             | 0.001     | 32                | 1000   |

en todos los modelos se utiliza **MSE** como función de pérdida, **Adam** como optimizador y una **paciencia** para el *Early Stop* de 50 épocas.

Para cada uno de los modelos entrenados se calcula el umbral  $\epsilon$  que permite distinguir entre instancias normales y anómalas a partir del error de reconstrucción. Dicho umbral se obtiene utilizando únicamente el conjunto de entrenamiento, siguiendo el criterio definido en la Ecuación 2.24. Los valores finales empleados por cada modelo se muestran en la Tabla 3.2.

Tabla 3.2: Valores de umbral  $\epsilon$  obtenidos para cada modelo.

| Modelo  | Umbral $\epsilon$ |
|---------|-------------------|
| AE (A)  | 4.618             |
| AE (B)  | 5.482             |
| AE (C)  | 6.137             |
| SAE (A) | 3.898             |
| SAE (B) | 4.351             |
| SAE (C) | 4.537             |
| CAE (A) | 2.996             |
| CAE (B) | 3.601             |
| CAE (C) | 3.319             |

Una vez definido el umbral para cada modelo, se analiza la distribución del error de reconstrucción sobre el conjunto de prueba para los **autocodificadores clásicos**. Los histogramas mostrados en las Figuras 3.1, 3.2 y 3.3 ilustran la distribución de dichos errores, distinguiendo los registros normales (en azul) de los anómalos (en rojo), así como la ubicación del umbral  $\epsilon$  en cada caso. Esta visualización muestra como el valor del umbral es insuficiente para lograr discernir correctamente entre ambas clases. Sin embargo, se cumple que las anomalías poseen un mayor error de reconstrucción que los datos normales.

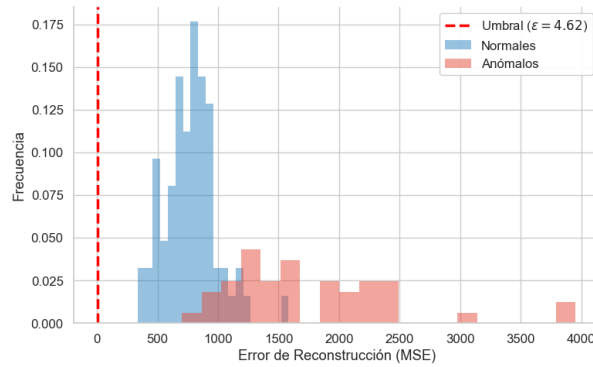


Figura 3.1: Distribución del error de reconstrucción para el autocodificador A.

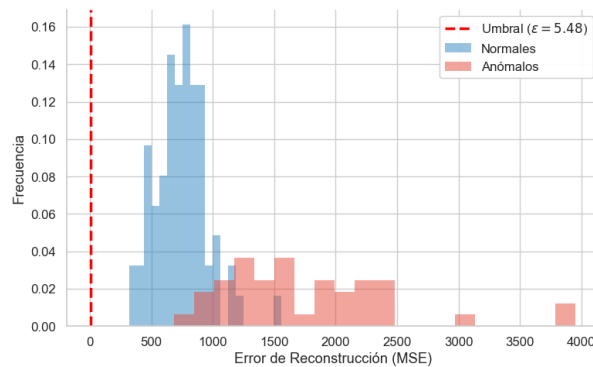


Figura 3.2: Distribución del error de reconstrucción para el autocodificador B.

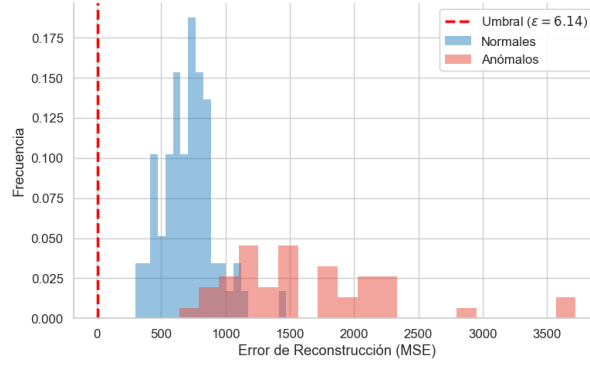


Figura 3.3: Distribución del error de reconstrucción para el autocodificador C.

Con el umbral computado para cada modelo se obtuvieron las métricas de la Tabla 3.3 donde siguiendo con los resultados anterior, se observa que el modelo no logra clasificar correctamente las instancias del conjunto de prueba utilizando el  $\epsilon$  calculado. Esto produce igualdad de valores en las métricas de todos los modelos.

Tabla 3.3: Métricas de rendimiento para cada modelo entrenado con los conjuntos A, B y C.

| Modelo  | <i>Accuracy</i> | <i>Precision</i> | <i>Recall</i> | <i>Specificity</i> | <i>Balanced Accuracy</i> | <i>F1<sub>score</sub></i> |
|---------|-----------------|------------------|---------------|--------------------|--------------------------|---------------------------|
| AE (A)  | 0.3772          | 0.3772           | 1.0           | 0.0                | 0.5                      | 0.5478                    |
| AE (B)  | 0.3772          | 0.3772           | 1.0           | 0.0                | 0.5                      | 0.5478                    |
| AE (C)  | 0.3772          | 0.3772           | 1.0           | 0.0                | 0.5                      | 0.5478                    |
| SAE (A) | 0.3772          | 0.3772           | 1.0           | 0.0                | 0.5                      | 0.5478                    |
| SAE (B) | 0.3772          | 0.3772           | 1.0           | 0.0                | 0.5                      | 0.5478                    |
| SAE (C) | 0.3772          | 0.3772           | 1.0           | 0.0                | 0.5                      | 0.5478                    |
| CAE (A) | 0.3772          | 0.3772           | 1.0           | 0.0                | 0.5                      | 0.5478                    |
| CAE (B) | 0.3772          | 0.3772           | 1.0           | 0.0                | 0.5                      | 0.5478                    |
| CAE (C) | 0.3772          | 0.3772           | 1.0           | 0.0                | 0.5                      | 0.5478                    |

Finalmente, con el objetivo de evaluar la capacidad discriminativa de cada arquitectura frente a instancias anómalas independientemente del umbral  $\epsilon$  seleccionado, se calculan las curvas ROC y sus correspondientes valores de AUC para todos los modelos entrenados. Para facilitar la comparación, los modelos se agrupan según el conjunto de entrenamiento utilizado (A, B y C), lo que permite analizar cómo varía el rendimiento ante distintos niveles de contaminación en el entrenamiento. Los resultados se presentan en las Figuras 3.4, 3.5 y 3.6.

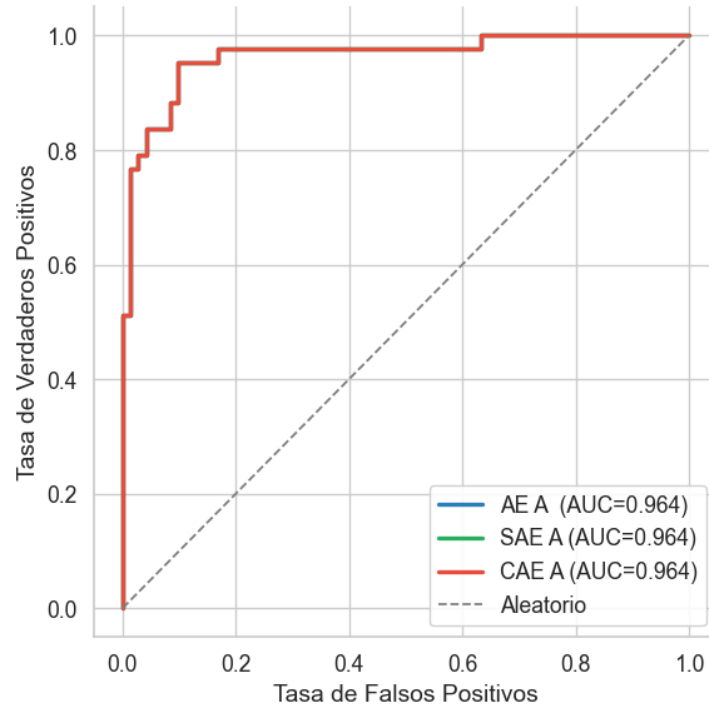


Figura 3.4: Curvas ROC y AUC obtenidos para los modelos AE, SAE y CAE entrenados con el Conjunto A (0 % de anomalías).

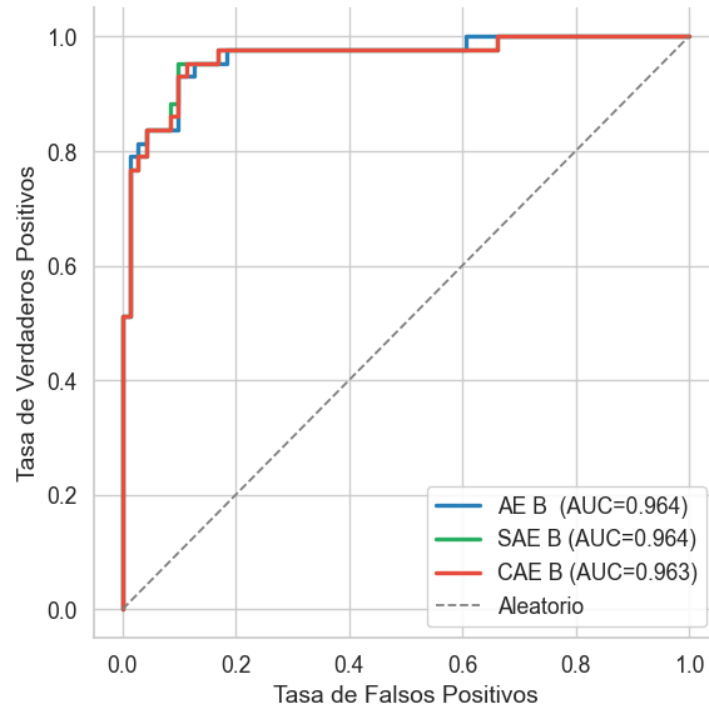


Figura 3.5: Curvas ROC y AUC obtenidos para los modelos AE, SAE y CAE entrenados con el Conjunto B (10 % de anomalías).



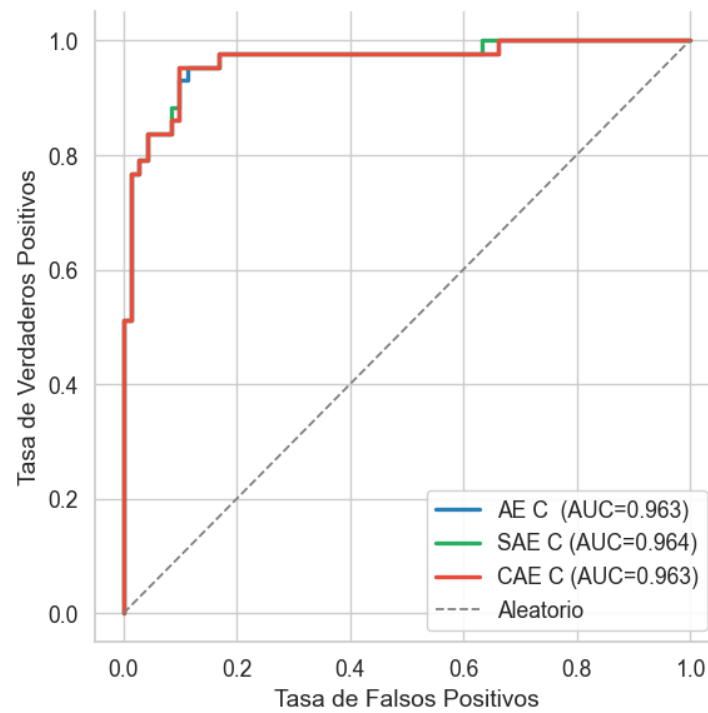


Figura 3.6: Curvas ROC y AUC obtenidos para los modelos AE, SAE y CAE entrenados con el Conjunto C (25 % de anomalías).



## Capítulo 4

# Conclusiones

El presente trabajo exploró el uso de autocodificadores (*Autoencoders*) para la detección de anomalías, aplicados sobre un conjunto de datos clínicos tras un proceso de análisis exploratorio y preprocesamiento. Se estudiaron tres variantes del modelo: clásico, *sparse* y *contractive* con el objetivo de evaluar cómo las diferencias en las arquitecturas afectan la capacidad del modelo para identificar patrones propios de los registros normales y distinguirlos de las anomalías.

A pesar de las expectativas iniciales, los resultados experimentales mostraron que ninguno de los modelos evaluados logró un desempeño satisfactorio. En todos los casos, las métricas de precisión, especificidad y exactitud fueron extremadamente bajas, mientras que el *recall* resultó artificialmente alto debido a que el umbral de decisión  $\epsilon$  llevó a clasificar todas las muestras como anómalas. Esto pone de manifiesto que el problema no radica exclusivamente en la arquitectura del autocodificador, sino en la definición del criterio de decisión utilizado para separar casos normales de anómalos.

En consecuencia, la principal conclusión de este trabajo es que la elección del umbral  $\epsilon$  constituye un elemento crítico que aún no ha sido resuelto de manera adecuada. El enfoque manual adoptado en esta primera experimentación no permitió capturar un valor óptimo que equilibrara sensibilidad y especificidad, por lo que es necesario avanzar hacia métodos más sistemáticos. Entre las líneas futuras de trabajo se destacan la búsqueda de estrategias automáticas basadas en análisis estadístico de la distribución del error de reconstrucción, técnicas de optimización, validación cruzada o métodos adaptativos que ajusten dinámicamente el umbral según el comportamiento del modelo.

Finalmente, se reconoce que, si bien los resultados no alcanzaron el rendimiento esperado, el estudio permitió profundizar en el entendimiento de los autocodificadores, su entrenamiento con datos no balanceados y la importancia de los criterios de decisión en tareas de detección de anomalías. Esto sienta una base sólida para continuar investigando y mejorando el enfoque, con el objetivo de lograr en futuras versiones un sistema robusto, confiable y clínicamente útil.



# Bibliografía

- [1] Dor Bank, Noam Koenigstein, and Raja Giryes. Autoencoders. *arXiv preprint arXiv:2003.05991*, 2020.
- [2] Roel Bouman and Tom Heskes. Autoencoders for anomaly detection are unreliable. *arXiv preprint arXiv:2501.13864*, 2025.
- [3] H Bourlard and Y Kamp. Auto-association by multilayer perceptrons and singular value decomposition. *Biol. Cybern*, 59:291–294, 1988.
- [4] Aristidis Diamantis, Emmanouil Magiorkinis, and Helen Koutselini. Fine-needle aspiration (fna) biopsy: historical aspects. *Folia histochemica et cytobiologica*, 47(2):191–197, 2009.
- [5] KS GGCS. Learning representations by back-propagating errors. *NATURE*, 323(9), 1986.
- [6] UCI Machine Learning and Ovsen. Breast cancer wisconsin (diagnostic) data set. <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data/data>. Kaggle Dataset. Accedido: 2025-11-27.
- [7] Ali Nawaz, Shehroz S. Khan, and Amir Ahmad. Ensemble of autoencoders for anomaly detection in biomedical data: A narrative review. *IEEE Access*, 12:17273–17289, 2024.
- [8] Asif Ahmed Neloy and Maxime Turgeon. A comprehensive study of auto-encoders for anomaly detection: Efficiency and trade-offs. *Machine Learning with Applications*, 17:100572, 2024.
- [9] Andrew Ng et al. Sparse autoencoder. *CS294A Lecture notes*, 72(2011):1–19, 2011.
- [10] Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot, and Yoshua Bengio. Contracting autoencoders. In *Proceedings of the International Conference on Machine Learning (ICML), Bellevue, WA, USA*, page 95, 2011.
- [11] William Nick Street, William H. Wolberg, and Olvi L. Mangasarian. Nuclear feature extraction for breast tumor diagnosis. In *Electronic imaging*, 1993.
- [12] Tomasz Walczyna, Damian Jankowski, and Zbigniew Piotrowski. Enhancing anomaly detection through latent space manipulation in autoencoders: A comparative analysis. *Applied Sciences*, 15(1):286, 2024.
- [13] William Wolberg, Olvi Mangasarian, Nick Street, and W. Street. Breast cancer wisconsin (diagnostic). UCI Machine Learning Repository, 1993. DOI: <https://doi.org/10.24432/C5DW2B>.